# Problem A. Test Submission

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

Return the product of the numbers $a$ and $b$ to begin your journey!

## Input

A single line containing integers $a$ and $b$, separated by a space.

## Output

The product of $a$ and $b$.

# Problem B. Black Box Breakdown

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

You find yourself stranded on a deserted island after a plane crash. The wreckage of the plane is scattered across the beach. You and the crew miraculously emerge from the landing unhurt, but now you must survive long enough for rescuers to help. There is only enough food and supplies to last a few days, so time is short. The island is also not entirely welcoming, with numerous ruins and potential threats endangering your party. You must act quickly to signal your location and status to search teams so that you can make it off alive!

While searching through the debris, you find the plane's black box, with crucial data logs from the flight's final moments.

In the logs, a string $s$ of $n$ lowercase characters holds vital information about the plane's flight path. However, only a few letters are relevant for your survival. You need to count how many times a letter in "pclassic" appears in the data.

The island is vast, and time is of the essence. Use your problem-solving skills to extract this lifesaving info!

## Input

The first line contains a single integer $n$ $(1 \le n \le 100)$ − the length of the string.

The second line contains the string $s$, consisting of only lowercase letters, of length $n$.

## Output

A single number, the number of times a character in "pclassic" occurs.

## Examples

| standard input | standard output |
|---|---|
| 10<br>helloworld | 3 |
| 15<br>pclassicissofun | 11 |

## Note

In the first example, the character 'l' is in "pclassic" and it appears 3 times in the input string.

In the second example, the characters 'p', 'c', 'l', 'a', 's', and 'i' are in "pclassic" and they appear a total of 11 times in the input string.

# Problem C. Critical Column Clearance

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

Key supplies are still inside the burning wreckage. You discover a half-buried control terminal and an old drone in the jungle. You decide to use the drone to retrieve what you need through a narrow slice of the plane. The terminal shows an $n$-by-$m$ grid mapping of the slice. Each column is a vertical slice of terrain and is formed from (possibly zero) consecutive 1's (denoting wreckage), followed by consecutive 0's (denoting air), and followed by consecutive 2's (denoting debris), in that order, from the top of the column to the bottom. The drone can only traverse through a location in the grid if it is open air.

The drone must fly in a straight line. Your goal is to find the height of the tallest gap of rows that is entirely open air (0's) in every column — this is where the drone can fly safely.

## Input

The first line contains two integers separated by a space, $n$ ($1 \le n \le 10^3$) − the number of rows, and $m$ ($1 \le m \le 10^3$) − the number of columns.

The next $n$ lines contain $m$ integers (either 0, 1, or 2) separated by spaces, representing the grid.

## Output

A single number, the size of the largest gap of rows which the drone can fly through.

## Examples

| standard input | standard output |
|---|---|
| 5 3<br>1 1 1<br>1 0 1<br>1 0 0<br>0 0 0<br>2 2 0 | 1 |
| 5 3<br>1 1 1<br>1 0 1<br>1 0 0<br>1 0 0<br>2 0 0 | 0 |

## Note

In the first example, the largest gap which goes through every column without hitting any debris is the 4th row only, so the largest gap has size 1.

In the second example, there is no gap through the first column, so the largest gap has size 0.

# Problem D. Rescue Radio Ratio

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

You recovered a damaged radio, and you're desperately trying to tune into a narrow safe band between two known interfering signals for each of $t$ potential rescuers' communication channels. Each channel has a lower interference signal at frequency $w/x$, and an upper interference signal at frequency $y/z$.

Radio stability is inversely related to the denominator of the frequency: the smaller the denominator $b$, the more stable the signal. All frequencies are simplified proper fractions (where numerator < denominator), and you must find a clean signal $a/b$ that sits strictly between them, not equal to either, and as stable with as small of a denominator as possible.

## Input

The first line contains $t$ ($1 \le t \le 100$) – the number of potential rescuer communication channels.

The next $t$ lines each contain numbers $w$, $x$, $y$, $z$, separated by spaces. The sum of $x * z$ over all $t$ tests is less than $10^6$.

## Output

$t$ lines, where the $i$-th line contains two numbers $a$ and $b$ (in that order) for the corresponding $i$-th rescuer communication channel, separated by spaces.

## Examples

| standard input | standard output |
|---|---|
| 1<br>1 2 3 4 | 2 3 |
| 1<br>11 13 12 13 | 6 7 |

## Note

In the first examle, $w = 1$, $x = 2$, $y = 3$, $z = 4$ so $w/x = 1/2$ and $y/z = 3/4$. Set $a = 2$ and $b = 3$, then $1/2 < 2/3 < 3/4$ and the value of the denominator $b$ is minimal.

In the second example, $w = 11$, $x = 13$, $y = 12$, $z = 13$ so $w/x = 11/13$ and $y/z = 12/13$. The pair $a = 6$, $b = 7$ satisfies $11/13 < 6/7 < 12/13$ and $b$ is minimal.

# Problem E. Scorched Sequence Solver

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

You discover a scorched notebook, part of the plane's systems manual, in the wreckage. Inside is a strange sequence of digits and symbols related to data on the island. It appears to be input to a primitive calculation engine.

Inside is a single string $s$ of length $n$, containing only digits (0-9) and symbols from the set $\{+, -, *, \$\}$. The \$ operation is defined in the margin of the manual as: $a\$b = a * b - a - b$. You need to evaluate it exactly as the system would, by performing operations strictly left to right, ignoring order of operations.

## Input

The first line contains a single integer $n$ ($1 \le n \le 2 * 10^5$) – the length of the string.

The second line contains $s$ – the string, consisting of alternating digits and operation symbols.

## Output

A single integer, the result of the operations.

## Examples

| standard input | standard output |
|---|---|
| 9<br>9+3*2-4$2 | 18 |
| 13<br>1-5$3+9$2+3*2 | -2 |

## Note

In the first example, calculate the operations left to right: $9 + 3 = 12$, $12 * 2 = 24$, $24 - 4 = 20$, $20\$2 = 20 * 2 - 20 - 2 = 18$.

In the second example: $1 - 5 = -4$, $-4\$3 = -11$, $-11 + 9 = -2$, $-2\$2 = -4$, $-4 + 3 = -1$, $-1 * 2 = -2$.

# Problem F. Missing Mask Mystery

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

You stumble upon a crumbling stone ruin in the island's overgrown interior. At its center: a circular depression, clearly meant to accept a single number. Above it, a worn inscription carved in an unfamiliar hand reads:

"Only those blessed with perfect form shall pass. But beware the Mask of Zero — only the true ones endure."

You realize this is a numerical riddle, likely guarding something important — perhaps a way off the island. You are given a number $n$. Among the sacred numbers from 1 to $n$, you must identify how many numbers $x$ satisfy the following conditions:

$x$ must be a perfect square. If you remove all 0 digits from its base-ten representation (the so-called Mask of Zero), the result must also be a perfect square.

## Input

The first line contains $n$ ($1 \le n \le 10^{12}$) – the upper bound on perfect squares you must find.

## Output

A single number, the number of $x$ which satisfy the property.

## Examples

| standard input | standard output |
|---|---|
| 100 | 10 |
| 1024 | 31 |

## Note

In the first example, each of the 9 perfect squares up to $n = 81$ have no zero digits, so clearly, each of those satisfies the property. 100 is a perfect square; 100 without all zeroes is 1, which is also a perfect square, so $n = 100$ is valid.

In the second example, 1024 is a perfect square; 1024 without all zeroes is 124, which is not a perfect square. Each of the 31 perfect squares before 1024 all satisfy the property.

# Problem G. Square Sand Signals

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

You and your team started laying out signal lines in the sand to attract rescuers. You dug $m$ trenches running north-south and $n$ running east-west, forming a grid visible from the air. Some of these trenches and intersections form exactly square signal zones — a sure sign of intelligent life.

Given that each trench is straight and no two are the same line, how many square-shaped zones can be spotted in the sand?

## Input

The first line contains two integers separated by a space, $n$ ($1 \le n \le 1000$) – the number of east-west lines, and $m$ ($1 \le m \le 1000$) – the number of north-south lines.

The second line contains $n$ integers separated by spaces, $y_i$ ($-10^6 \le y_i \le 10^6$) – the coordinates of the east-west lines.

The third line contains $m$ integers separated by spaces, $x_i$ ($-10^6 \le x_i \le 10^6$) – the coordinates of the north-south lines.

## Output

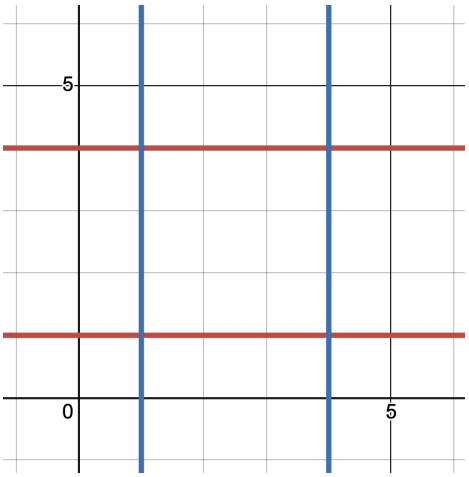
A single number, the number of squares formed.

## Examples

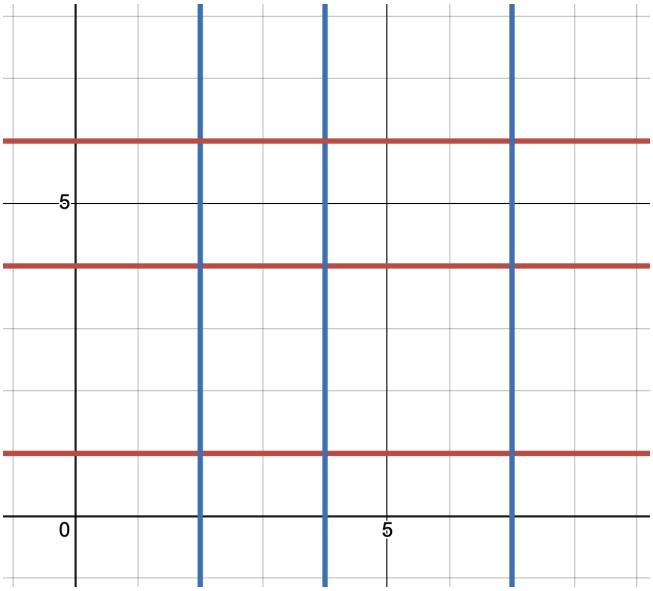
| standard input | standard output |
|---|---|
| 2 2<br>1 4<br>1 4 | 1 |
| 3 3<br>1 4 6<br>2 4 7 | 3 |

## Note

In the first example, the four lines form the boundaries of a square with vertices at $(1, 1)$, $(1, 4)$, $(4, 4)$, $(4, 1)$.




In the second example, three squares are formed, with side lengths 2, 3, 5.


5
0
5

# Problem H. Covert Camouflage Crafting

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 2 seconds |
| Memory limit: | 256 megabytes |

To avoid attracting unwanted attention from hostile forces on the island, you must craft a camouflage pattern that blends in with $n$ unique natural textures you've observed — each texture encoded as an integer, representing its binary pattern of light and shadow.

You want your camouflage to minimize the visual contrast with the surroundings. For each natural texture $a_i$, the contrast is defined as the number of differing bits between your pattern and the texture. (Before comparing, all patterns are padded with leading zeroes to match in length.)

However, some patterns are off-limits — there are $m$ banned designs known to be easily spotted. Your mission: find a non-banned camouflage pattern $x$ that results in the least total contrast with the environment such that it does not match any banned design $b_j$. If there are multiple such patterns, choose the smallest one.

## Input

The first line contains two integers separated by a space, $n$ ($1 \le n \le 10^5$) – the number of patterns to minimize contrast with, and $m$ ($1 \le m \le 10^5$) – the number of banned camouflage patterns.

The second line contains $n$ integers in decimal representation, $a_i$ ($0 \le a_i < 2^{31}$) – representing the patterns to minimize contrast with.

The third line contains $m$ integers in decimal representation, $b_j$ ($0 \le b_j < 2^{31}$) – representing the banned patterns.

## Output

A single integer, $x$, in decimal representation.

## Examples

| standard input | standard output |
|---|---|
| 3 1<br>1 2 3<br>2 | 3 |
| 2 2<br>7 8<br>7 8 | 0 |

## Note

In the first example, $1_{10} = 01_2$, $2_{10} = 10_2$, $3_{10} = 11_2$. Choosing $x = 3_{10} = 11_2$ has the smallest sum of differences: $1 + 1 + 0 = 2$.

In the second example, $7_{10} = 0111_2$, $8_{10} = 1000_2$. Choosing $x = 0_{10} = 0000_2$ has the smallest sum of differences: $3 + 1 = 4$, and it is the minimum of all integers who have a sum of differences of 4.

# Problem I. Colorful Corridor Conundrum

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 2 seconds |
| Memory limit: | 256 megabytes |

While searching for more water, you find yourself in a mysterious underground labyrinth with $N$ rooms. The labyrinth consists of a network of $M$ passages (undirected paths) between rooms, each marked with a specific color that is one of $C$ possible colors. In order to escape the labyrinth, you must travel from your starting room $s$ to the exit room $t$. You can do this in two ways:

Change your path's color: If you need to switch from color $c$ to another color $c'$, it will cost you an additional amount of energy $x$ to make the switch.

Travel along a colored path: If you're currently following a path of color $c$, you can move between any rooms that are connected to the current room with paths of color $c$ for a cost $y$.

You can start on any path color. Your goal is to find the minimum energy needed to get from your starting room to the exit room. Output $-1$ if it is impossible to reach the exit room from the starting room.

## Input

The first line contains seven integers $N, M, C, x, y, s, t$ ($1 \le N \le 10^5$, $1 \le C \le M \le 10^5$, $1 \le x, y \le 10^6$, $1 \le s, t \le 10^5$). The next $M$ lines contain 3 integers $u_i, v_i, t_i$ $1 \le u_i, v_i \le N, 1 \le c_i \le C$), indicating that there is an passage from room $u_i$ to $v_i$ with color $c_i$.

## Example

| standard input | standard output |
|---|---|
| 6 6 3 4 1 2 5<br>1 2 1<br>2 3 1<br>3 1 2<br>3 4 2<br>4 5 3<br>2 6 2 | 11 |

# Problem J. Forecast Factor Frenzy

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

The island's only weather station has malfunctioned, and now you're relying on a bizarre, old forecasting algorithm left by a reclusive meteorologist. According to their notes, the "storm stability index" for a period of $n$ days is computed by the following mysterious ritual:

For each day $i$ from 1 to $n$, calculate the number of ways the wind patterns could have shifted — represented by $i!$ — and take the product of all these values across all $n$ days. We then calculate the number of divisors of this product.

Unfortunately, the numbers get huge fast, so to keep things manageable, the number of divisors of the final value must be reported modulo $10^9 + 7$ — the only prime the station's software recognizes.

## Input

The first line contains $t$ ($1 \le t \le 10^4$), the number of tests.

The next $t$ lines contain an integer $n$ ($1 \le n \le 10^7$).

The sum of $n$ over all test cases will not exceed $10^7$.

## Output

For each test, output the number of divisors of the product of the number of ways the wind patterns could have shifted modulo $10^9 + 7$.

## Example

| standard input | standard output |
|---|---|
| 5<br>1<br>2<br>3<br>4<br>10 | 1<br>2<br>6<br>18<br>28080 |

# Problem K. Wandering Waypoint Woes

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 2 seconds |
| Memory limit: | 256 megabytes |

While exploring the dense island interior, your group discovers a network of ancient trails connecting n waypoints — a vast system of winding paths built long ago. Each path between two waypoints has a roughness level, a number representing how uneven or difficult it is to walk. The network is such that there's exactly one unique trail between any two waypoints (some say the trails follow the natural roots of the island's massive trees).

Now, you're tasked with $q$ missions — each asking you to travel from one waypoint $s$ to another $t$. For each mission, you must report how unbalanced the terrain is along the trail you'd walk.

- For each mission from $s$ to $t$, compute the average roughness across all paths along the way
- Then, for each path, compute how far its roughness deviates from that average
- Average all these deviations

## Input

First line contains two integers, $n$ and $q$ ($1 \le n, q \le 10^5$).

The next $n-1$ lines contain three integers $u$, $v$, $w$ ($1 \le u, v \le n$, $1 \le w \le 10^9$), the trails and their roughness levels.

The next $q$ lines contain two integers $s$ and $t$ ($1 \le s, t \le n$, $s \ne t$), the missions.

## Output

For each mission, output its unbalanced level.

## Example

| standard input | standard output |
|---|---|
| 6 5<br>1 3 1<br>1 4 3<br>2 4 4<br>3 6 2<br>4 5 6<br>1 3<br>1 6<br>5 6<br>5 2<br>3 2 | 0.0000000000<br>0.5000000000<br>1.5000000000<br>1.0000000000<br>1.1111111111 |

## Note

The average roughness of the mission from 1 to 6 is $(1+2)/2 = 3/2$.

Therefore the unbalanced level is $(|1-3/2| + |2-3/2|)/2 = 1/2$.

The average roughness of the mission from 3 to 2 is $(1+3+4)/3 = 8/3$.

Therefore the unbalanced level is $(|1-8/3| + |3-8/3| + |4-8/3|)/3 = 10/9$.

# Problem L. Shelter Shuffle Struggle

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

With storms sweeping across the island, your group decides to build $n$ round shelters, each with a radius based on the materials gathered — there are $n$ such shelters.

For safety, these shelters must be placed along a flat stretch of cleared land represented by a number line from 0 to $x$. Each shelter must be placed with its center at an integer location within this range. Due to their round shape, shelters can't overlap, even partially.

There's one more constraint: one shelter must be anchored exactly at point 0 — your team's command center. Your task is to count how many valid ways you can arrange all $n$ shelters, placing each one at an integer center in $[0, x]$, ensuring no overlaps and satisfying the anchoring rule. Since the number of arrangements can be large, return your answer modulo $10^9 + 7$.

## Input

The first line contains two integers, $n$ and $x$ ($1 \le n \le 10^5$, $1 \le x \le 10^6$)

The next line contains $n$ integers $r_i$ ($1 \le r_i \le 10^5$)

## Output

Output the number of arrangements modulo $10^9 + 7$

## Examples

| standard input | standard output |
|---|---|
| 1 3<br>4 | 1 |
| 2 3<br>2 2 | 0 |
| 2 4<br>1 2 | 4 |
| 4 20<br>1 4 6 2 | 200 |

# Problem M. Investigative Identity Inquiry

| | |
|---|---|
| Input file: | `standard input` |
| Output file: | `standard output` |
| Time limit: | 1 second |
| Memory limit: | 256 megabytes |

Now that the rescuers have established a line of communication with you, they need to verify that it's really you over radio. You and your group have a secret code. The code was embedded in a string s that you prepared beforehand — a long, cryptic sequence of letters or numbers.

To confirm your identity, the rescuers give a query pattern and $q$ tests, each with four indices of the code string: $a$, $b$, $c$, and $d$. For each test, you must provide the frequency the pattern $p$ appears as a substring in a distinct starting location in the combined string formed by concatenating the substring from $a$ to $b$ (inclusive) and the substring from $c$ to $d$ (inclusive)?

If you can correctly answer the query, the rescuers will know you are indeed the survivors.

## Input

The first line contains a string $s$ ($1 \le |s| \le 10^5$)

The next line contains a string $p$ ($1 \le |p| \le 10^5$)

The next line contains an integer $q$ ($1 \le q \le 10^5$)

The next $q$ lines contain four integers $a_i$, $b_i$, $c_i$, $d_i$ ($1 \le a_i \le b_i \le |s|$, $1 \le c_i \le d_i \le |s|$)

## Output

For each query, output the number of times $p$ appears in $s[a_i, b_i] + s[c_i, d_i]$

## Example

| standard input | standard output |
|---|---|
| abcdabcd<br>bc<br>5<br>2 3 6 7<br>2 3 6 6<br>1 2 3 4<br>3 4 7 8<br>1 8 1 8 | 2<br>1<br>1<br>0<br>4 |

## Note

The first combined string is "bcbc", in which bc appears 2 times.

The second combined string is "bcb", in which bc appears 1 time.

The third combined string is "abcd", in which bc appears 1 time.

The fourth combined string is "bccd", in which bc appears 1 time.